

intel_mgmt_gen.vr

\$VCS_HOME/doc/examples/nativetestbench/openvera/rvm/host_if/intel_mgmt/intel_mgmt_gen.vr

```
/*
 *
 * Author:      Fabian Delguste
 * File:        $RCSfile: intel_mgmt_gen.vr,v $
 * Revision:    $Revision: 1.11 $
 * Date:        $Date: 2004/01/19 15:37:11 $
 *
 ****
 *
 * Basic Transaction Verification Module aimed at
 * creating read() and write() transactions based
 * on Intel management Interface
 *
 ****
 */

typedef class intel_mgmt_gen;
typedef class intel_mgmt_scenario;

class intel_mgmt_gen_callbacks extends rvm_xactor_callbacks {
    // Callback method called after each scenario generated
    virtual task post_scenario_gen_t(intel_mgmt_gen    gen,
                                     intel_mgmt_scenario scenario)

    {
    }

    // Callback method used to modify transaction descriptor
    // prior to sending over
    virtual task pre_tr_fwd_t(intel_mgmt_gen    gen,
                              intel_mgmt_trans tr,
                              var bit          drop)

    {
    }
}

class intel_mgmt_scenario {
    // Factory instance used to build the scenario items
    intel_mgmt_trans randomized_tr;

    integer stream_id;
    integer scenario_id;
```

```

static bit [31:0] NULL = 0;
rand bit [31:0] kind;
rand bit [31:0] length;
rand intel_mgmt_trans items[$];
local integer next_idx;
constraint intel_mgmt_scenario_basic {
    solve kind before length;
    items.size() == length;
    if (kind == NULL) length == 1;
}
constraint valid_scenarios {
    kind == NULL;
}
task new() {
    this.randomized_tr = new;
}
protected task resize_items(integer n) {
    while (this.items.size() < n) {
        intel_mgmt_trans tr;

        this.randomized_tr.object_id = this.items.size();
        cast_assign(tr, this.randomized_tr.copy());
        this.items.push_back(tr);
    }
}
task pre_randomize()
{
    // Flush the scenario list because they
    // have been forwarded to the output channel.
    // We need new instances to avoid modifying the
    // same ones from the previous scenario...
    this.items.delete();

    // Pre-allocate enough scenario items
    // for the maximum-length scenario
    this.randomized_tr.stream_id = this.stream_id;
    this.randomized_tr.scenario_id = this.scenario_id;
    this.resize_items(1);
    this.next_idx = 0;
}
task post_randomize()

```

```

{
    // Trim the items because the solver only solves
    // for the constrained size, not the actual size!
    while (this.items.size() > this.length) {
        void = this.items.pop_back();
    }
}

virtual function intel_mgmt_trans next_item()
{
    if (next_idx >= this.items.size()) {
        next_item = null;
    } else {
        next_item = this.items[next_idx++];
    }
}
}

class intel_mgmt_gen extends rvm_xactor {
    integer scenario_id;

    // Blueprint (factory) instance
    intel_mgmt_scenario randomized_sc;
    // Output rvm_channel: this is where transactions
    // to be sent end up
    intel_mgmt_trans_channel out_trans;
    // Stop after having generated the specified number of scenarios
    // 0 == never stop (default)
    integer stop_after_n_scenarios;

    task new(string instance,
             integer stream_id,
             intel_mgmt_trans_channel out_trans = null) {

        super.new("Intel Mgmt Transaction Generator", instance);

        if (out_trans == null) {
            out_trans = new({this.log.get_name(), " Output Channel"}, instance);
        }
        this.out_trans = out_trans;

        this.randomized_sc = new;
        this.scenario_id = 0;
        this.stop_after_n_scenarios = 0;
    }
}

```

```

}

// Method aimed at generating transactions
// (derived from the rvm_xactor)
protected virtual task main_t();

// Reset the transaction generator
virtual task reset_xactor(integer rst_type = 0);
// Directed interface
virtual task write(bit [4:0] addr, bit [7:0] data);
virtual function bit [7:0] read(bit [4:0] addr);

}

task intel_mgmt_gen::main_t() {

    fork
        super.main_t();
    join none
    while (1) {
        intel_mgmt_trans tr;
        bit                drop = 0;
        if (this.stop_after_n_scenarios > 0 &&
            this.scenario_id >= this.stop_after_n_scenarios) {
            this.stop_xactor();
        }
        super.wait_if_stopped_t();
        this.randomized_sc.stream_id  = this.stream_id;
        this.randomized_sc.scenario_id = this.scenario_id++;
        // Randomize the scenario
        void = this.randomized_sc.randomize();
        // Report the scenario via a callback
        rvm_00_callback(intel_mgmt_gen_callbacks,
                        post_scenario_gen_t(this, this.randomized_sc));

        // Forward the scenario to the output channel
        for (tr = this.randomized_sc.next_item();
            tr != null;
            tr = this.randomized_sc.next_item()) {

            // Give a chance to modify object
            rvm_00_callback(intel_mgmt_gen_callbacks,

```

```

        pre_tr_fwd_t(this, tr, drop));
    if (!drop) {
        out_trans.put_t(tr);
    }
}
}

// Reset the transaction generator
task intel_mgmt_gen::reset_xactor(integer rst_type = 0)
{
    super.reset_xactor(rst_type);
    this.scenario_id = 0;
}

// Directed interface
task intel_mgmt_gen::write(bit [4:0] addr,
                           bit [7:0] data)
{
    intel_mgmt_trans tr = new;
    tr.kind = intel_mgmt_trans::WRITE;
    tr.addr = addr;
    tr.data = data;
    this.out_trans.put_t(tr);
    void = tr.notify.wait_for_t(tr.ENDED);
}

function bit [7:0] intel_mgmt_gen::read(bit [4:0] addr)
{
    intel_mgmt_trans tr = new;
    tr.kind = intel_mgmt_trans::READ;
    tr.addr = addr;
    this.out_trans.put_t(tr);
    void = tr.notify.wait_for_t(tr.ENDED);
    read = tr.data;
}

```